SLOT1
INTRODUCTION TO FLUTTER

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Overview of Flutter
  - What is Flutter - Why Flutter
  - Flutter architecture ?
- Dart Language Basics
  - Introduction to Dart and its role in Flutter
- Development Environments
  - DartPad, VS Code, Android Studio
- Setting Up Flutter
  - Installing SDK & Android Studio
  - Configuring Emulator
- Creating & Running Your First App

🎯 Mục tiêu tổng quát của Module 1

Sau khi hoàn thành Module 1, sinh viên có thể:

- Hiểu Flutter là gì, vì sao Flutter được sử dụng phổ biến trong phát triển ứng dụng đa nền tảng
- Nắm được vai trò của Dart trong hệ sinh thái Flutter
- Thiết lập đầy đủ môi trường phát triển Flutter
- Tạo, chạy và hiểu cấu trúc một ứng dụng Flutter cơ bản

1️⃣ Overview of Flutter

🔷 Khái niệm & nội dung chính

- **What is Flutter**
  - Flutter là SDK mã nguồn mở của Google để phát triển ứng dụng **đa nền tảng** (Android, iOS, Web, Desktop) từ **một codebase duy nhất**.
- **Why Flutter**
  - Hiệu năng gần native, Hot Reload, UI nhất quán, cộng đồng lớn.
- **Flutter Architecture**
  - Kiến trúc 3 lớp:
    - Framework (Widgets, Rendering, Animation)
    - Engine (Skia, Dart VM)
    - Embedder (Android, iOS, Web…)

🎯 Mục tiêu học tập

- Hiểu được **bức tranh tổng thể** của Flutter trong ngành phát triển phần mềm
- Phân biệt Flutter với các framework khác (React Native, Native Android/iOS)
- Nhận thức được cách Flutter render UI và vì sao Flutter không dùng WebView

2️⃣ Dart Language Basics

🔷 Khái niệm & nội dung chính

- **Introduction to Dart**

    - Dart là ngôn ngữ lập trình do Google phát triển, được thiết kế tối ưu cho UI và reactive programming.
- **Role of Dart in Flutter**
    - Dart là ngôn ngữ **duy nhất** dùng để viết Flutter app
    - Hỗ trợ OOP, async/await, null safety

🎯 **Mục tiêu học tập**

- Hiểu vì sao Flutter **bắt buộc dùng Dart**
- Nắm được những đặc điểm cốt lõi của Dart để sẵn sàng học Widget
- Có thể đọc hiểu các đoạn code Dart cơ bản trong Flutter

3️⃣ Development Environments

🔷 **Khái niệm & nội dung chính**

- **DartPad**
    - Môi trường online để thử Dart/Flutter nhanh, không cần cài đặt
- **VS Code**
    - Editor nhẹ, phổ biến, nhiều extension Flutter/Dart mạnh mẽ
- **Android Studio**
    - IDE chính thức, hỗ trợ Emulator, Debug, Profiler

🎯 **Mục tiêu học tập**

- Biết được **các lựa chọn môi trường phát triển** Flutter
- Lựa chọn được công cụ phù hợp với cấu hình máy và mục đích học tập
- Hiểu vai trò của IDE trong quy trình phát triển ứng dụng

4️⃣ Setting Up Flutter

🔷 **Khái niệm & nội dung chính**

- **Installing Flutter SDK**
    - Cài Flutter SDK, thiết lập biến môi trường, kiểm tra bằng `flutter doctor`
- **Installing Android Studio**
    - Cài Android SDK, Platform tools
- **Configuring Emulator**
    - Tạo và chạy Android Emulator hoặc kết nối thiết bị thật

🎯 **Mục tiêu học tập**

- Hoàn thiện **môi trường phát triển Flutter sẵn sàng chạy app**
- Hiểu được các thành phần cần thiết để Flutter hoạt động
- Biết cách tự kiểm tra và xử lý lỗi cài đặt cơ bản

5️⃣ Creating & Running Your First App

🔷 **Khái niệm & nội dung chính**

- Tạo project Flutter mới (`flutter create`)
- Cấu trúc thư mục Flutter cơ bản:
    - `lib/`, `main.dart`, `pubspec.yaml`
- Chạy ứng dụng trên Emulator / thiết bị thật
- Làm quen với **Hot Reload**

🎯 **Mục tiêu học tập**

- Tạo và chạy thành công **ứng dụng Flutter đầu tiên**
- Hiểu vai trò của file `main.dart`
- Có trải nghiệm thực tế đầu tiên với Flutter workflow

📚 **Tài liệu tham khảo uy tín (3–5 nguồn)**

1. **Flutter Official Documentation**
   👉 https://docs.flutter.dev
   Tài liệu chính thức, đầy đủ nhất từ Google (bắt buộc đọc)

Book: **Mastering Flutter Learn to develop Flutter apps for iOS, Android, desktop and web (Kevin Moore)**

2. **Dart Language Tour (Official)**
   👉 https://dart.dev/guides/language/language-tour
   Học Dart từ cơ bản đến nâng cao, phù hợp cho Flutter
3. **Flutter Crash Course – Traversy Media (YouTube)**
   Video nhập môn trực quan, dễ tiếp cận mới
4. **Flutter in Action – Eric Windmill (Book, Manning)**
   Sách nền tảng tốt để hiểu kiến trúc và tư duy Flutter
5. **Official Flutter YouTube Channel**
   Video ngắn, chất lượng cao về kiến trúc, widget, best practices

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1️⃣ Nhận thức đúng về Flutter (Conceptual Foundation)

🔷 **Khái niệm quan trọng**

- Flutter là **SDK UI đa nền tảng**, không phải framework web
- Flutter render UI bằng **engine riêng (Skia)** → không dùng WebView
- **Everything is a Widget**: UI trong Flutter được xây dựng từ widget

🔷 **Kỹ năng cần nắm**

- Giải thích được Flutter khác gì Native Android / iOS / React Native
- Nhận biết được khi nào Flutter là lựa chọn phù hợp

❌ **Lỗi thường gặp**

- Nhầm Flutter là "viết 1 lần, chạy mọi nơi" nhưng không cần tối ưu
- Nghĩ Flutter chỉ dùng cho mobile

✅ **Best practices**

- Hiểu Flutter như một **hệ sinh thái** (Dart + Framework + Engine)
- Luôn gắn Flutter với tư duy UI component-based

2️⃣ Hiểu kiến trúc Flutter ở mức nền tảng (Architecture Awareness)

🔷 **Khái niệm quan trọng**

- Flutter gồm 3 lớp: **Framework – Engine – Embedder**
- Widget → Element → RenderObject (chưa cần chi tiết, chỉ cần khái niệm)

🔷 **Kỹ năng cần nắm**

- Mô tả luồng render UI cơ bản trong Flutter
- Hiểu vì sao Flutter có hiệu năng cao

❌ **Lỗi thường gặp**

- Học Flutter nhưng không biết app được vẽ như thế nào
- Nhầm Flutter dùng native UI component của Android/iOS

✅ **Best practices**

- Chỉ cần **hiểu tổng quan**, không đào sâu quá sớm
- Ghi nhớ: Flutter **tự vẽ UI**, không phụ thuộc OS UI

3️⃣ Dart – Ngôn ngữ bắt buộc phải nắm nền tảng

🔷 **Khái niệm quan trọng**

- Dart là ngôn ngữ **chính thức & duy nhất** của Flutter
- Hướng đối tượng, null safety, async/await

🔷 **Kỹ thuật cần thành thạo (mức nhập môn)**

- Khai báo biến (`var`, `final`, `const`)
- Hàm, class, constructor
- Arrow function
- Hiểu `main()` là entry point

❌ **Lỗi thường gặp**

- So sánh Dart theo kiểu "giống Java/JS nên không cần học"
- Không phân biệt `final` và `const`

- Bị lỗi null nhưng không hiểu nguyên nhân

✅ **Best practices**

- Học Dart **song song với Flutter**, không tách rời
- Tập thói quen đọc error message của Dart compiler

4️⃣ Làm chủ môi trường phát triển (Tooling Proficiency)

🔷 **Khái niệm quan trọng**

- IDE ≠ Flutter SDK
- Emulator ≠ Flutter app

🔷 **Kỹ năng cần thành thạo**

- Biết dùng **VS Code hoặc Android Studio**
- Biết sử dụng **DartPad** để test nhanh code
- Chạy và dừng Emulator đúng cách

❌ **Lỗi thường gặp**

- Cài Flutter nhưng IDE không nhận SDK
- Emulator quá nặng → hiểu nhầm Flutter "chạy chậm"

✅ **Best practices**

- Ưu tiên VS Code cho máy yếu
- Luôn kiểm tra plugin Flutter & Dart đã được cài

5️⃣ Cài đặt Flutter & xử lý lỗi cơ bản (Setup Mastery)

🔷 **Khái niệm quan trọng**

- Flutter SDK + Android SDK + Emulator = hệ sinh thái hoàn chỉnh
- `flutter doctor` là công cụ kiểm tra sức khỏe hệ thống

🔷 **Kỹ năng cần thành thạo**

- Chạy `flutter doctor` và đọc kết quả
- Hiểu các warning/error phổ biến (SDK, license, path)

❌ **Lỗi thường gặp**

- Bỏ qua lỗi `flutter doctor`
- Không hiểu ý nghĩa các dấu ❌ ⚠️ ✅

✅ **Best practices**

- Luôn fix lỗi `flutter doctor` trước khi viết code
- Ghi nhớ: "Không chạy được app → quay lại flutter doctor"

6️⃣ Tạo & chạy app Flutter đầu tiên (Hands-on Core Skill)

🔷 **Khái niệm quan trọng**

- `main.dart` là điểm bắt đầu chương trình
- `runApp()` khởi chạy UI
- `MaterialApp` là app wrapper phổ biến

🔷 **Kỹ năng cần thành thạo**

- Tạo project bằng `flutter create`
- Chạy app trên Emulator / thiết bị thật
- Thực hiện **Hot Reload**

❌ **Lỗi thường gặp**

- Sửa code nhưng quên save → Hot Reload không chạy
- Nhầm Hot Reload và Hot Restart

✅ **Best practices**

- Thường xuyên Hot Reload để quan sát thay đổi
- Đọc kỹ log khi app crash

7️⃣ **Tư duy học Flutter đúng ngay từ đầu (Learning Mindset)**

🔷 **Khái niệm quan trọng**

- Flutter học **từ widget nhỏ → app lớn**
- Không học Flutter theo kiểu "copy & paste"

🔷 **Kỹ năng cần rèn**

- Đọc code mẫu và hiểu từng dòng
- Debug lỗi thay vì hỏi ngay

❌ **Lỗi thường gặp**

- Học Flutter chỉ để "chạy được app"
- Sợ lỗi đỏ, không đọc log

✅ **Best practices**

- Ưu tiên **hiểu – không thuộc**
- Tập giải thích lại code bằng lời của mình

⭐ **Tóm tắt ngắn gọn – Sinh viên bắt buộc phải nắm được sau bài học**

✅ Giải thích được Flutter là gì và vì sao dùng Flutter

✅ Hiểu vai trò của Dart trong Flutter

✅ Cài đặt và kiểm tra môi trường Flutter hoàn chỉnh

✅ Tạo và chạy được app Flutter đầu tiên

✅ Biết đọc lỗi cơ bản và tự sửa lỗi setup.

I. PHẦN 1 – HỌC THUẬT CỐT LÕI (WHY & WHAT)

🎯 **Mục tiêu**

- Xây dựng **nền tảng nhận thức đúng**
- Tránh học Flutter theo kiểu "mò"

Nội dung học thuật trọng tâm

- Flutter là gì? Vì sao Flutter tồn tại?
- Flutter Architecture (mức khái niệm)
- Dart và vai trò trong Flutter
- Flutter workflow (code → build → run → render)

II. PHẦN 2 – THỰC HÀNH NHỎ KIỂM CHỨNG TỪNG NỘI DUNG

🔑 Nguyên tắc: *mỗi khái niệm lý thuyết phải có ít nhất 1 hành động thực tế để kiểm chứng*

🔷 **Thực hành 1: Kiểm chứng Flutter là "Everything is a Widget"**

🧪 **Nhiệm vụ**

- Tạo Flutter app
- Thay đổi nội dung Text, màu sắc, layout
- Quan sát Hot Reload

🎯 **Kiến thức được kiểm chứng**

- UI được xây từ widget
- Thay đổi code → UI đổi ngay
- Flutter không cần rebuild toàn bộ app

❓ **Câu hỏi phản tư**

- Vì sao Flutter hot reload nhanh hơn native?
- Widget khác gì view trong Android?

🔷 **Thực hành 2: Kiểm chứng vai trò của Dart**

🧪 **Nhiệm vụ**

- Viết hàm Dart đơn giản trong `main.dart`
- Dùng biến `final`, `const`
- Quan sát lỗi khi dùng sai null safety

🎯 **Kiến thức được kiểm chứng**

- Dart là ngôn ngữ điều khiển toàn bộ Flutter
- Compiler Dart phát hiện lỗi sớm

❓ **Câu hỏi phản tư**

- Vì sao Flutter không dùng Java/Kotlin?
- Dart giúp Flutter tối ưu điều gì?

🔷 **Thực hành 3: Kiểm chứng môi trường & tooling**

🧪 **Nhiệm vụ**

- Chạy `flutter doctor`
- Cố tình tạo lỗi (thiếu emulator, SDK)
- Quan sát log & cách fix

🎯 **Kiến thức được kiểm chứng**

- Flutter phụ thuộc môi trường
- Tooling ảnh hưởng trực tiếp đến năng suất

❓ **Câu hỏi phản tư**

- Vì sao lỗi setup nguy hiểm trong dự án thật?
- Nếu làm nhóm, cần thống nhất điều gì?

III. PHẦN 3 – BÀI TẬP TỔNG HỢP (CAPSTONE MINI TASK)

🔥 Mục tiêu: buộc sinh viên **kết hợp toàn bộ nội dung Module 1**

🧠 **BÀI TẬP TỔNG HỢP ĐỀ XUẤT**

📌 **Bài tập: "Flutter Starter App"**

📝 **Yêu cầu**

Sinh viên phải:

- Tạo project Flutter mới
- Chạy app trên Emulator
- Sửa UI hiển thị:
  - Tên sinh viên
  - Mô tả ngắn: “Why I learn Flutter”
- Sử dụng:
  - Ít nhất 2 widget cơ bản
  - Dart variable (`final`)
- Chụp màn hình app đang chạy

🎯 **Kiến thức được đánh giá**

Nội dung	Được áp dụng
Flutter overview	✓
Dart basics	✓
Tooling	✓
Setup environment	✓
First app	✓

IV. ĐÁNH GIÁ THEO NĂNG LỰC (OUTCOME-BASED)

✅ **Sau Module 1, sinh viên đạt được:**

- **Nhận thức**: Hiểu Flutter & Dart là gì, vì sao dùng
- **Kỹ năng**: Tự setup & chạy app Flutter
- **Tư duy**: Biết kiểm chứng lý thuyết bằng thực hành
- **Thái độ**: Không sợ lỗi, biết đọc log

III. ỨNG DỤNG THỰC TIỄN

I. ỨNG DỤNG THỰC TIỄN: 3 TÌNH HUỐNG DỰ ÁN CỤ THỂ

🔷 Tình huống 1: Startup cần xây dựng MVP nhanh cho ứng dụng Mobile

📌 Bối cảnh thực tế

- Một startup nhỏ (3–5 dev) cần xây dựng **MVP (Minimum Viable Product)** cho ứng dụng đặt lịch dịch vụ
- Thời gian gấp (1–2 tháng), ngân sách hạn chế
- Yêu cầu chạy được trên **Android & iOS**

⚠️ Vấn đề gặp phải

- Nếu dùng native: phải viết **2 codebase**
- Đội ngũ nhỏ, không đủ nhân lực Android + iOS
- Cần demo sớm cho nhà đầu tư

🛠️ Kỹ thuật từ Module 1 được áp dụng

- **Flutter overview & why Flutter**
  - Quyết định dùng Flutter để viết **1 codebase đa nền tảng**
- **Dart language basics**
  - Dùng Dart để xây UI, xử lý logic cơ bản
- **Development environment**
  - VS Code + Android Emulator cho dev nhanh
- **Creating & running first app**
  - Tạo prototype UI trong vài ngày đầu

🎯 Mục tiêu áp dụng

- Rút ngắn thời gian phát triển MVP
- Có bản chạy thực tế để demo sớm

✅ Kết quả

- MVP hoàn thành đúng hạn
- Nhận được feedback sớm từ người dùng & nhà đầu tư

📘 Bài học rút ra

👉 **Hiểu đúng Flutter ngay từ đầu giúp ra quyết định công nghệ đúng, tiết kiệm chi phí & thời gian.**

🔷 Tình huống 2: Nhóm sinh viên / đội dự án nội bộ triển khai app quản lý nội bộ

📌 Bối cảnh thực tế

- Một đội IT nội bộ cần app quản lý:
  - Chấm công
  - Thông báo nội bộ
- Thành viên mới **chưa từng học Flutter**

⚠️ Vấn đề gặp phải

- Môi trường dev mỗi người khác nhau → lỗi setup liên tục
- Không biết bắt đầu từ đâu với Flutter

🛠️ Kỹ thuật từ Module 1 được áp dụng

- **Setting up Flutter**
  - Chuẩn hóa cài đặt Flutter SDK, Android Studio
  - Dùng `flutter doctor` kiểm tra đồng bộ môi trường

- **Development Environments**
  - Thống nhất dùng VS Code + Emulator
- **First App**
  - Cùng tạo app mẫu để kiểm tra môi trường trước khi code thật

🎯 **Mục tiêu áp dụng**

- Giảm thời gian setup ban đầu
- Tránh lỗi môi trường khi làm việc nhóm

✅ **Kết quả**

- 100% thành viên chạy được app Flutter trong tuần đầu
- Không mất thời gian debug lỗi môi trường trong giai đoạn sau

📘 **Bài học rút ra**

👉 **Setup & tooling là bước "nhỏ nhưng sống còn" trong dự án thật. Không làm tốt sẽ phá hỏng tiến độ.**

🔷 **Tình huống 3: Công ty chuyển từ Mobile Native sang Flutter**

📌 **Bối cảnh thực tế**

- Công ty đã có app Android native
- Muốn mở rộng sang iOS nhưng:
  - Chi phí cao
  - Khó bảo trì song song 2 codebase

⚠️ **Vấn đề gặp phải**

- Code trùng lặp
- Thời gian release lâu
- Bug fix không đồng bộ

🛠️ **Kỹ thuật từ Module 1 được áp dụng**

- **Flutter architecture**
  - Hiểu Flutter render UI độc lập → UI nhất quán
- **Dart language**
  - Đào tạo dev Android học Dart cơ bản
- **Creating first Flutter app**
  - Xây app mẫu để đánh giá hiệu năng

🎯 **Mục tiêu áp dụng**

- Đánh giá khả năng chuyển đổi công nghệ
- Chuẩn bị cho migration dài hạn

✅ **Kết quả**

- Quyết định dùng Flutter cho các module mới
- Giảm ~40% thời gian phát triển tính năng mới

📘 **Bài học rút ra**

👉 **Không cần biết Flutter sâu ngay, nhưng phải hiểu đúng kiến trúc để đánh giá tính phù hợp.**

II. CASE STUDY: DỰ ÁN / CÔNG TY ÁP DỤNG FLUTTER THÀNH CÔNG

🧮 **Case Study: Alibaba Group – Ứng dụng Flutter trong Mobile App**

📌 **Bối cảnh**

- Alibaba cần:
    - Phát triển nhanh nhiều tính năng UI
    - Đảm bảo hiệu năng cao
    - Đồng bộ Android & iOS

⚠️ **Vấn đề**

- Native development quá chậm cho tốc độ business
- Khó maintain UI phức tạp

🛠️ **Cách họ áp dụng các kỹ thuật nền tảng**

- **Flutter Architecture**
    - Tận dụng khả năng render UI bằng engine riêng
- **Dart**
    - Dart dễ học cho đội Android/iOS
- **Hot Reload**
    - Giảm thời gian chỉnh UI từ phút xuống giây
- **Tooling chuẩn**
    - Chuẩn hóa môi trường Flutter cho toàn đội

🎯 **Mục tiêu**

- Tăng tốc phát triển
- Giảm bug UI
- Đồng bộ trải nghiệm người dùng

✅ **Kết quả**

- Flutter được dùng cho nhiều module trong app
- Thời gian phát triển UI giảm đáng kể
- Đội dev thích nghi nhanh

📘 **BÀI HỌC RÚT RA (CỰC KỲ QUAN TRỌNG)**

✔ Hiểu kiến trúc Flutter giúp tự tin dùng trong dự án lớn

✔ Dart không khó nếu học đúng nền tảng

✔ Tooling & workflow ảnh hưởng trực tiếp đến năng suất

✔ Flutter mạnh nhất khi dùng đúng mục tiêu: UI nhanh, đa nền tảng

III. THÔNG ĐIỆP

Module 1 không phải để "viết app hay"
mà để:

- Ra quyết định công nghệ đúng
- Setup môi trường chuẩn
- Hiểu nền tảng để học các module sau **không bị vỡ**

IV. MỘT SỐ NỘI DUNG CHI TIẾT

1️⃣ OVERVIEW OF FLUTTER

1.1 Flutter là gì? (What is Flutter)

🔷 Khái niệm

Flutter là một **UI SDK (Software Development Kit)** mã nguồn mở do **Google** phát triển, cho phép xây dựng ứng dụng **đa nền tảng** (Android, iOS, Web, Desktop) **từ một codebase duy nhất**, sử dụng ngôn ngữ **Dart**.

👉 Flutter **không phải**:

- Framework web
- Công cụ chuyển web thành mobile
- Wrapper của native UI

🔷 Flutter dùng để làm gì?

- Xây dựng **UI động, tương tác cao**
- Phát triển app nhanh với **Hot Reload**
- Đồng bộ UI & logic giữa các nền tảng

🔷 Vì sao Flutter quan trọng?

- Giải quyết bài toán **chi phí + thời gian + nhân lực**
- Đảm bảo UI nhất quán
- Hiệu năng gần native

1.2 Vì sao chọn Flutter? (Why Flutter)

🔷 So sánh nhanh

Tiêu chí	Flutter	Native	React Native
Codebase	1	2	1
Hiệu năng	Cao	Rất cao	Trung bình
UI nhất quán	✓	❌	❌
Hot Reload	✓	❌	✓

🔷 Khi nào nên dùng Flutter?

- MVP, startup
- App có UI phức tạp
- Đội ngũ nhỏ

🔷 Khi KHÔNG nên dùng Flutter?

- App phụ thuộc SDK native quá sâu
- Game 3D nặng (Unity phù hợp hơn)

1.3 Flutter Architecture (CỰC KỲ QUAN TRỌNG)

🔷 **Kiến trúc 3 lớp**

```
Flutter App (Dart)
│
├── Framework (Widgets, Animation, Material)
│
├── Engine (Skia, Dart VM, Text Rendering)
│
└── Embedder (Android, iOS, Web, Desktop)
```

🔷 **Nguyên lý hoạt động**

- Flutter **tự vẽ UI**
- Không dùng UI component native
- Engine Skia render trực tiếp pixel

🔷 **Vì sao Flutter nhanh?**

- Không bridge JS ↔ Native
- UI được kiểm soát hoàn toàn bởi Flutter

❌ **Hiểu sai phổ biến**

- "Flutter dùng UI của Android/iOS"
- "Flutter giống React Native"

👉 **Sai hoàn toàn**

✅ **Best practices**

- Hiểu kiến trúc ở mức **tổng quan**
- Chưa cần đào sâu RenderObject ở Module 1

🔗 **Liên kết kiến thức**:

- Kiến trúc → Widget
- Widget → Dart
- Dart → UI logic

2️⃣ DART LANGUAGE BASICS

2.1 Dart là gì?

🔷 **Khái niệm**

Dart là ngôn ngữ lập trình do Google phát triển, tối ưu cho:

- UI
- Reactive programming
- Hiệu năng cao

👉 **Flutter = Dart + Framework**

🔷 **Vai trò của Dart trong Flutter**

- Viết UI

- Quản lý state
- Xử lý logic
- Giao tiếp API

2.2 Các chủ điểm Dart nền tảng cần nắm

🔷 **Entry point**

```
void main() {
  print("Hello Flutter");
}
```

🔷 **Biến & kiểu dữ liệu**

```
var name = "Flutter";
final version = 3;
const pi = 3.14;
```

- `final`: runtime constant
- `const`: compile-time constant

🔷 **Hàm**

```
int sum(int a, int b) => a + b;
```

🔷 **Class (OOP)**

```
class User {
  final String name;
  User(this.name);
}
```

🔷 **Null Safety (rất quan trọng)**

```
String? title; // có thể null
String name = "Flutter"; // không được null
```

❌ **Lỗi thường gặp**

- Nhầm `final` với `const`
- Không hiểu dấu ?

✅ **Cách phòng tránh**

- Luôn đọc error message
- Dùng DartPad test nhanh

🔗 **Liên kết kiến thức**:

- Dart → Widget
- Dart → main.dart

3️⃣ DEVELOPMENT ENVIRONMENTS

3.1 DartPad

🔷 **Vai trò**

- Test Dart/Flutter **online**
- Không cần cài SDK

👉 Phù hợp:

- Học Dart
- Demo nhanh trên lớp

3.2 VS Code

🔷 **Vai trò**

- Editor nhẹ
- Extension Flutter/Dart mạnh

🔷 **Plugin cần cài**

- Flutter
- Dart

3.3 Android Studio

🔷 **Vai trò**

- IDE chính thức
- Quản lý Emulator, SDK

❌ **Lỗi phổ biến**

- Cài Flutter SDK nhưng IDE không nhận
- Quên cài plugin Dart

✅ **Best practices**

- Máy yếu → VS Code
- Máy mạnh → Android Studio

4️⃣ SETTING UP FLUTTER

4.1 Cài Flutter SDK

🔷 **Các bước**

1. Tải Flutter SDK
2. Giải nén
3. Thêm vào PATH
4. Kiểm tra:

```
flutter doctor
```

4.2 Android Studio & Emulator

🔷 **Cấu hình Emulator**

- Chọn Pixel
- Android API phù hợp
- RAM ≥ 2GB

❌ **Lỗi thường gặp**

- Emulator không chạy
- Thiếu Android license

✅ **Cách khắc phục**

```
flutter doctor --android-licenses
```

🔷 **Best practices**

- Fix **100% flutter doctor** trước khi code

5️⃣ CREATING & RUNNING YOUR FIRST APP

5.1 Tạo project

```
flutter create my_first_app
cd my_first_app
flutter run
```

5.2 Cấu trúc project

```
lib/
 └── main.dart
pubspec.yaml
```

5.3 Code minh họa đầu tiên

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: const Text("Hello Flutter")),
        body: const Center(
          child: Text("My First Flutter App"),
        ),
      ),
    );
  }
}
```

🔷 **Giải thích**

- `runApp()` → khởi động app
- `MaterialApp` → wrapper

- `Widget` → UI

🔥 **Hot Reload**
- Save file → UI đổi ngay
- Không mất state

❌ **Lỗi thường gặp**
- Quên save
- Nhầm Hot Reload vs Restart

✅ **Best practices**
- Luôn quan sát log
- Thử sửa nhỏ để hiểu widget

TỔNG KẾT SƯ PHẠM
Module 1 không nhằm tạo app phức tạp
mà tạo tư duy, nền tảng và sự tự tin
Sinh viên **hiểu sâu Module 1** sẽ:
- Không bị "vỡ" khi học Widget, Layout, State
- Không sợ lỗi
- Biết mình đang làm gì

📘 **TỪ ĐIỂN THUẬT NGỮ NỀN TẢNG FLUTTER (FOUNDATION GLOSSARY)**

1️⃣ **UI (User Interface)**
🔷 **Định nghĩa**
UI (Giao diện người dùng) là toàn bộ **những gì người dùng nhìn thấy và tương tác** trong ứng dụng: văn bản, nút bấm, hình ảnh, bố cục, màu sắc, animation.
🔷 **Trong Flutter**
- UI được xây dựng **hoàn toàn bằng Widget**
- Flutter **tự vẽ UI**, không dùng UI component của hệ điều hành

🔷 **Ý nghĩa thực tiễn**
- Flutter tập trung rất mạnh vào UI → học Flutter là học **tư duy UI**

2️⃣ **Widget (CỐT LÕI NHẤT CỦA FLUTTER)**
🔷 **Định nghĩa**
Widget là **đơn vị cấu thành nhỏ nhất của UI trong Flutter**.
👉 Trong Flutter: **Everything is a Widget**
🔷 **Ví dụ Widget**
- Text
- Button
- Image
- Row, Column
- Scaffold, AppBar

🔷 **Đặc điểm**

- Widget **không giữ trạng thái UI lâu dài**
- Widget mô tả **UI sẽ trông như thế nào**, không phải nó đang hiển thị ra sao

🔷 **Phân loại cơ bản**

- **StatelessWidget**: không thay đổi theo thời gian
- **StatefulWidget**: UI thay đổi khi dữ liệu thay đổi

🔷 **Lỗi hiểu sai phổ biến**

- Nghĩ widget = view (Android) → ❌
- Nghĩ widget là object hiển thị trực tiếp → ❌

👉 Widget là **bản mô tả UI**

3️⃣ Layout

🔷 **Định nghĩa**

Layout là cách **sắp xếp các widget con** trong không gian UI.

🔷 **Trong Flutter**

- Layout cũng là widget
- Ví dụ:
  - Row (ngang)
  - Column (dọc)
  - Stack (chồng)
  - Padding, Expanded

🔷 **Ý nghĩa**

- Flutter layout dựa trên **constraint-based layout**
- Hiểu layout giúp tránh lỗi vỡ UI

4️⃣ State

🔷 **Định nghĩa**

State là dữ liệu có thể thay đổi theo thời gian và **ảnh hưởng trực tiếp đến UI**.

🔷 **Ví dụ**

- Số đếm
- Trạng thái đăng nhập
- Dữ liệu từ API

🔷 **Trong Flutter**

- Khi state thay đổi → UI được rebuild
- Flutter là **declarative UI framework**

🔷 **Hiểu sai phổ biến**

- Nghĩ state = biến bình thường → ❌
- Quên gọi `setState()` → UI không đổi

5️⃣ Wrapper

🔷 **Định nghĩa**

Wrapper là widget dùng để **bao bọc (wrap)** các widget khác nhằm:

- Cung cấp cấu hình
- Thêm hành vi chung

- Áp dụng theme, routing

◆ **Ví dụ**

- MaterialApp
- Scaffold
- Center

◆ **Ý nghĩa**

- Wrapper giúp **tái sử dụng & cấu trúc app rõ ràng**

6 Codebase

◆ **Định nghĩa**

Codebase là toàn bộ mã nguồn của một dự án.

◆ **Trong Flutter**

- Flutter dùng **1 codebase**
- Deploy ra nhiều nền tảng

◆ **Ý nghĩa**

- Giảm chi phí
- Dễ bảo trì

7 UI SDK (User Interface Software Development Kit)

◆ **Định nghĩa**

UI SDK là bộ công cụ dùng để **xây dựng giao diện người dùng**.

◆ **Flutter là UI SDK vì:**

- Cung cấp widget
- Cung cấp công cụ render UI
- Không chỉ là thư viện

8 SDK Native

◆ **Định nghĩa**

Native SDK là SDK chính thức của nền tảng:

- Android SDK (Java/Kotlin)
- iOS SDK (Swift/Objective-C)

◆ **So với Flutter**

Native SDK	Flutter
UI native	UI custom
2 codebase	1 codebase

9 Framework (Flutter Framework)

◆ **Định nghĩa**

Framework là lớp cung cấp:

- Widget
- API
- Logic UI

◆ **Thành phần chính**

- Widgets
- Animation
- Material / Cupertino

🔷 **Vai trò**

- Dev tương tác trực tiếp với framework

1️⃣0️⃣ Engine

🔷 **Định nghĩa**

Engine là tầng thấp xử lý:

- Render
- Text
- Runtime

🔷 **Thành phần**

- **Skia**: vẽ đồ họa
- **Dart VM**: chạy Dart code
- **Text Rendering**: hiển thị chữ

🔷 **Ý nghĩa**

- Flutter không phụ thuộc UI native

1️⃣1️⃣ Embedder

🔷 **Định nghĩa**

Embedder là lớp kết nối Flutter Engine với hệ điều hành.

🔷 **Ví dụ**

- Android Embedder
- iOS Embedder
- Web Embedder

🔷 **Vai trò**

- Kết nối camera, file, sensor

1️⃣2️⃣ Hot Reload

🔷 **Định nghĩa**

Hot Reload cho phép cập nhật UI **ngay lập tức** khi thay đổi code mà không restart app.

🔷 **Ý nghĩa**

- Tăng tốc phát triển
- Test UI nhanh

🔷 **Giới hạn**

- Không áp dụng cho thay đổi cấu trúc app lớn

1️⃣3️⃣ DartPad

🔷 **Định nghĩa**

DartPad là môi trường online để chạy Dart/Flutter.

🔷 **Vai trò**

- Học Dart
- Demo nhanh

- Test logic

🔗 SƠ ĐỒ LIÊN KẾT TƯ DUY (MENTAL MODEL)

```
Flutter (UI SDK)
│
├── Dart (ngôn ngữ)
├── Framework (Widget, Layout, State)
├── Engine (Render, VM)
├── Embedder (OS)
│
└── Tools (Hot Reload, DartPad)
```

🎓 THÔNG ĐIỆP

Hiểu thuật ngữ = hiểu cách Flutter vận hành
Hiểu cách vận hành = học Flutter không bị mù mờ
EVERYTHING IS A WIDGET (FLUTTER)

I. MỤC TIÊU BÀI HỌC

Sau bài giảng này, sinh viên có thể:

- Hiểu **bản chất triết lý "Everything is a Widget"** trong Flutter
- Phân biệt **Widget – Element – RenderObject** ở mức nền tảng
- Nhận diện **các nhóm widget** và vai trò của chúng
- Hiểu **vòng đời hiển thị UI** trong Flutter
- Áp dụng tư duy widget để **đọc – viết – phân tích code Flutter đúng cách**

II. VẤN ĐỀ ĐẶT RA: VÌ SAO FLUTTER KHÁC?

Trong Android/iOS truyền thống:

- UI = XML / Storyboard
- Logic = Java/Kotlin / Swift
- UI và logic **tách rời**

👉 Flutter chọn cách tiếp cận khác:
UI = Code = Widget Tree

III. ĐỊNH NGHĨA CỐT LÕI: WIDGET LÀ GÌ?

1. Định nghĩa học thuật

Widget là:
Một **bản mô tả bất biến (immutable description)** của một phần giao diện người dùng tại một thời điểm nhất định.

👉 Widget **không phải** là thứ đang hiển thị trên màn hình.

2. Diễn giải dễ hiểu

- Widget giống như **bản thiết kế UI**
- Flutter dùng widget để:
  - So sánh UI cũ – mới
  - Quyết định có cần vẽ lại hay không

IV. EVERYTHING IS A WIDGET – HIỂU ĐÚNG TRIẾT LÝ

Trong Flutter:

- Text là widget
- Button là widget
- Padding là widget
- Layout (Row, Column) là widget
- App cũng là widget

👉 **Không có khái niệm UI nào nằm ngoài widget**

V. WIDGET TREE – CẤU TRÚC XƯƠNG SỐNG

Flutter app được tổ chức dưới dạng **cây widget (Widget Tree)**

Ví dụ đơn giản:

```
MaterialApp
└── Scaffold
├── AppBar
└── Center
└── Text
```

Ý nghĩa:

- Mỗi widget chỉ biết **con của nó**
- UI được xây dựng từ trên xuống dưới

VI. 3 LỚP QUAN TRỌNG: WIDGET – ELEMENT – RENDEROBJECT

1. Widget

- Mô tả UI
- Immutable
- Được tạo lại rất часто

2. Element

- Đại diện cho **widget trong cây UI đang chạy**
- Quản lý lifecycle

3. RenderObject

- Thực sự **vẽ pixel lên màn hình**
- Xử lý layout, size, position

👉 Sinh viên **chưa cần code RenderObject**, nhưng phải biết nó tồn tại.

VII. PHÂN LOẠI WIDGET CƠ BẢN

1. StatelessWidget

- Không có state thay đổi
- UI cố định theo input

Ví dụ:

```
class MyText extends StatelessWidget {
@override
Widget build(BuildContext context) {
return const Text("Hello Flutter");
}
}
```

2. StatefulWidget

- Có state
- UI thay đổi theo thời gian

```
class Counter extends StatefulWidget {
@override
State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
int count = 0;

@override
Widget build(BuildContext context) {
return Text("Count: $count");
}
}
```

VIII. BUILD METHOD – TRÁI TIM CỦA WIDGET

1. build() là gì?

- Hàm mô tả UI
- Có thể được gọi lại **rất nhiều lần**

2. Hiểu sai phổ biến

- build() chỉ chạy 1 lần ❌
- build() = render ❌

👉 build() chỉ **trả về widget tree mới**

IX. DECLARATIVE UI – TƯ DUY CỐT LÕI

Flutter dùng **Declarative UI**:
"UI trông như thế nào khi state = X"
Thay vì:

- "Thay đổi UI từng bước"

Flutter:

- Thay đổi state → rebuild widget

X. WRAPPER WIDGET – TƯ DUY BAO BỌC

1. Định nghĩa

Wrapper widget là widget dùng để:

- Bao widget khác
- Cung cấp cấu hình chung

2. Ví dụ

- MaterialApp
- Scaffold
- Padding
- Center

👉 Flutter code thường **lồng nhiều widget**

XI. LỖI SINH VIÊN RẤT HAY GẶP

❌ Nghĩ widget là UI hiển thị trực tiếp ❌ Sợ widget lồng nhiều tầng ❌ Viết logic nặng trong build() ❌ Không hiểu vì sao UI rebuild

XII. BEST PRACTICES KHI LÀM VIỆC VỚI WIDGET

✅ Chia nhỏ widget ✅ Widget nhỏ – dễ test – dễ đọc ✅ build() chỉ để build UI ✅ Hiểu widget tree trước khi code

XIII. LIÊN KẾT VỚI CÁC CHỦ ĐỀ KHÁC

- Widget → Layout
- Widget → State
- Widget → Navigation
- Widget → Performance

👉 **Hiểu widget tốt = học Flutter nhanh gấp 3 lần**

XIV. THÔNG ĐIỆP CHỐT

"Flutter không khó – chỉ khó khi không hiểu Widget"
Everything is a Widget không phải slogan, mà là **mô hình tư duy bắt buộc** khi học Flutter

V. KINH NGHIỆM THỰC TIỄN, TIPS

Module 1: Introduction to Flutter

Tài liệu này **bổ sung chiều sâu thực tiễn** cho Module 1, giúp sinh viên:

- Tránh các lỗi "nhập môn nhưng rất tốn thời gian"
- Hiểu khoảng cách giữa **lý thuyết** và **dự án thật**
- Hình thành thói quen lập trình Flutter **đúng chuẩn ngay từ đầu**

1️⃣ OVERVIEW OF FLUTTER (What & Why Flutter)

✅ Kinh nghiệm thực tế

- Trong dự án thật, Flutter **thường được chọn vì tốc độ phát triển**, không phải vì "viết 1 lần chạy mọi nơi" tuyệt đối
- Flutter phát huy sức mạnh nhất ở **UI phức tạp, thay đổi nhanh**, MVP, sản phẩm cần test thị trường sớm

📌 Điều phát sinh ngoài lý thuyết

- Một số plugin chỉ hỗ trợ Android/iOS → vẫn cần hiểu native cơ bản
- Build size ban đầu có thể lớn hơn native

🔍 Tình huống thực tế

Startup dùng Flutter làm app chính, nhưng module camera nâng cao phải viết native → tích hợp qua platform channel

⚠️ Lỗi thường gặp

Lỗi: Nghĩ Flutter thay thế hoàn toàn native

- ❌ Nguyên nhân: hiểu Flutter là "all-in-one"
- ✅ Khắc phục: xác định Flutter là **UI layer chính**, native là bổ trợ

💡 Mẹo thực tế

- Luôn hỏi: *Flutter giúp giải quyết bài toán gì của dự án?*
- Dùng Flutter cho UI, không ép Flutter làm mọi thứ

🛡️ Best Practices

- Đánh giá yêu cầu dự án trước khi chọn Flutter
- Không over-promise khả năng cross-platform

2️⃣ DART LANGUAGE BASICS

✅ Kinh nghiệm thực tế

- Dart dễ học nhưng **null safety gây lỗi nhiều nhất cho người mới**
- Viết Dart "thoải mái" giống JS/Java → dễ tạo bug logic

📌 Điều phát sinh ngoài lý thuyết

- Code Dart compile được nhưng logic sai
- Async/await gây bug race condition

⚠️ Lỗi thường gặp

❌ Lỗi 1: Lạm dụng `var`

var data = fetchData(); // khó đọc, khó debug
Khắc phục: dùng kiểu rõ ràng
❌ **Lỗi 2: Không hiểu null safety**
String name;
print(name.length); // crash
Khắc phục: hiểu `?`, `!`, giá trị mặc định

💡 **Mẹo lập trình**
- Dùng DartPad để test logic nhanh
- Đọc error message Dart (rất rõ ràng)

🛡️ **Best Practices**
- Hạn chế `dynamic`
- Ưu tiên `final`
- Code Dart rõ nghĩa > code ngắn

3️⃣ **DEVELOPMENT ENVIRONMENTS (DartPad, VS Code, Android Studio)**
✅ **Kinh nghiệm thực tế**
- 80% lỗi sinh viên gặp trong 2 tuần đầu là **lỗi môi trường**
- IDE không quyết định trình độ, nhưng ảnh hưởng năng suất

📌 **Điều phát sinh ngoài lý thuyết**
- Máy yếu → emulator lag → hiểu nhầm Flutter chậm

⚠️ **Lỗi thường gặp**
Lỗi: IDE không nhận Flutter SDK
- ❌ Nguyên nhân: PATH sai, thiếu plugin
- ✅ Khắc phục: kiểm tra Settings + flutter doctor

💡 **Mẹo**
- VS Code cho học tập
- Android Studio cho debug sâu

🛡️ **Best Practices**
- Thống nhất IDE trong nhóm
- Không code khi flutter doctor còn lỗi

4️⃣ **SETTING UP FLUTTER & EMULATOR**
✅ **Kinh nghiệm thực tế**
- Setup là bước "không tạo ra sản phẩm" nhưng quyết định thành bại dự án

📌 **Điều phát sinh ngoài lý thuyết**
- Lỗi license Android
- Emulator không đủ RAM

⚠️ Lỗi thường gặp

❌ Lỗi: Emulator không chạy

- Nguyên nhân: thiếu virtualization
- Khắc phục: bật VT-x / SVM trong BIOS

💡 Mẹo

- Test app trên **thiết bị thật** nếu emulator lag

🛡️ Best Practices

- Fix 100% flutter doctor
- Ghi lại checklist setup cho team

5️⃣ CREATING & RUNNING YOUR FIRST APP

✅ Kinh nghiệm thực tế

- App chạy được ≠ hiểu app
- Sinh viên thường "qua nhanh" bước này → thiếu nền

📌 Điều phát sinh ngoài lý thuyết

- Hot Reload không áp dụng cho mọi thay đổi

⚠️ Lỗi thường gặp

❌ Lỗi: UI không cập nhật

- Nguyên nhân: quên save / dùng sai Hot Reload
- Khắc phục: Hot Restart

💡 Mẹo

- Thử thay đổi nhỏ (Text, Color) để hiểu widget

🛡️ Best Practices

- Đọc kỹ main.dart
- Hiểu MaterialApp, Scaffold trước khi học layout

🔄 LIÊN HỆ THỰC TẾ DỰ ÁN

Trong dự án thật, team Flutter mạnh không phải vì code nhiều widget, mà vì **hiểu nền tảng, tránh lỗi sớm, và giữ codebase sạch**.

🎓 THÔNG ĐIỆP CHỐT

Module 1 quyết định 70% tốc độ học Flutter về sau

Học kỹ Module 1 giúp:

- Ít bug
- Dễ debug
- Dễ mở rộng
- Không sợ kiến trúc phức tạp

EXERCISE

📋 Tên dự án

MyFirstFlutterApp – Personal Info & Counter App

🎯 Mục tiêu sư phạm

Sau dự án, sinh viên có thể:

- Hiểu triết lý **Everything is a Widget**
- Phân biệt **StatelessWidget vs StatefulWidget**
- Hiểu luồng chạy Flutter: `main()` → `runApp()` → Widget Tree
- Biết cách **tách widget – clean code**
- Sử dụng **Hot Reload / Debug**

👉 Đây là **project mức cơ bản–trung bình**, đủ để mô phỏng quy trình dự án thật.

2. Chức năng chính của ứng dụng

- Hiển thị thông tin cá nhân (Tên, nghề nghiệp, mô tả ngắn)
- Hiển thị avatar (ảnh hoặc icon)
- Nút tăng / giảm số đếm (Counter)
- Cập nhật UI theo State
- Chạy được trên Android Emulator

3. Công nghệ & công cụ sử dụng

- **Flutter SDK (stable)**
- **Dart**
- **VS Code** (code chính)
- **Android Studio** (SDK + Emulator)
- Flutter Material Widgets

4. Cấu trúc thư mục dự án (chuẩn Flutter)

```
my_first_flutter_app/
│
├── lib/
│   ├── main.dart
│   ├── screens/
│   │   └── home_screen.dart
│   └── widgets/
│   ├── hello_widget.dart
│   ├── profile_card.dart
│   └── counter_widget.dart
│
├── pubspec.yaml
└── README.md
```

👉 Giải thích:

- `main.dart`: khởi tạo app, MaterialApp
- `screens`: các màn hình
- `widgets`: widget tái sử dụng
- `assets`: tài nguyên tĩnh

5. Danh sách các bước thực hiện (tóm tắt)

- ☐ Cài Flutter SDK
- ☐ Cài Android Studio + Emulator
- ☐ Cấu hình VS Code
- ☐ Tạo project Flutter
- ☐ Chạy app mặc định
- ☐ Phân tích main.dart
- ☐ Tạo StatelessWidget đầu tiên
- ☐ Tạo StatefulWidget (Counter)
- ☐ Tạo ProfileCard (StatelessWidget)
- ☐ Tạo HomeScreen & gắn toàn bộ widget
- ☐ Debug – Hot Reload

6. Bước 1 – Cài đặt Flutter SDK

1.1. Yêu cầu tối thiểu

- Hệ điều hành: Windows 10+ / macOS / Linux
- RAM: ≥ 8GB (khuyến nghị)
- Dung lượng trống: ≥ 15GB

1.2. Kiểm tra

- Máy bật được Android Studio
- Có quyền cài phần mềm

✅ **Hoàn thành khi**: Máy đáp ứng yêu cầu, không lỗi hệ thống

Thực hiện

1. Truy cập flutter.dev → Download
2. Giải nén Flutter SDK
3. Thêm Flutter vào PATH

2.1. Tải Flutter

1. Mở trình duyệt
2. Truy cập: https://flutter.dev
3. Chọn **Get Started → Install**
4. Chọn hệ điều hành
5. Tải file ZIP

2.2. Giải nén Flutter

- Giải nén vào:
  - Windows: `C:\flutter`
  - macOS/Linux: `~/flutter`

⚠️ KHÔNG đặt trong Program Files

2.3. Thêm Flutter vào PATH

Windows:

- Search → Environment Variables
- Edit Path → Add: `C:\flutter\bin`

macOS/Linux:
export PATH="$PATH:$HOME/flutter/bin"

2.4. Kiểm chứng Flutter đã cài

Mở **Terminal / CMD**, gõ:
flutter --version

✅ **Thành công khi**: thấy phiên bản Flutter hiển thị

❌ **Lỗi thường gặp**:
- `flutter is not recognized` → sai PATH

Kết quả mong đợi
- Flutter SDK: ✓
- Dart SDK: ✓

⚠️ Lỗi thường gặp:
- PATH chưa đúng → flutter command not found

7. Bước 2 – Cài đặt Android Studio & Emulator

Thực hiện
- Cài Android Studio
- Cài:
  - Android SDK
  - Android SDK Platform-Tools
  - Android Emulator

Tạo Emulator
- Device: Pixel 4
- API: 30+ (Android 11 trở lên)

3.1. Tải Android Studio
- Truy cập: https://developer.android.com/studio
- Download → Install

3.2. Cài các thành phần BẮT BUỘC

Trong Android Studio → SDK Manager:
- Android SDK Platform
- Android SDK Platform-Tools
- Android Emulator

3.3. Tạo máy ảo (Emulator)
1. Mở **Device Manager**
2. Create Device
3. Chọn: Pixel 4
4. Chọn Android API 30+
5. Finish

3.4. Kiểm chứng Android

flutter doctor

✅ **Thành công khi**:

👉 Android toolchain: ✓

8. Bước 3 – Cấu hình VS Code
Cài extension

- Flutter
- Dart

4.1. Cài VS Code

- https://code.visualstudio.com

4.2. Cài extension
Trong VS Code → Extensions:

- Flutter
- Dart

4.3. Kiểm chứng

- Ctrl + Shift + P
- Gõ: `Flutter: New Project`

✅ Có Flutter command

9. Bước 4 – Tạo project Flutter
flutter create my_first_flutter_app
cd my_first_flutter_app
code .

5.1. Tạo project
Mở Terminal:
flutter create my_first_flutter_app
5.2. Mở project
cd my_first_flutter_app
code .
✅ Thấy thư mục `lib/`, `pubspec.yaml`

10. Bước 5 – Chạy app mặc định
6.1. Bật Emulator

- Android Studio → Device Manager → Start

6.2. Chạy app
flutter run
6.3. Kiểm chứng

- App hiển thị màn hình Counter
- Bấm nút + → số tăng

🎯 **Ý nghĩa học thuật**:

- App đang chạy trên Flutter Engine
- UI là Widget

🎯 Sinh viên cần hiểu: **đây là StatefulWidget**

11. Bước 6 – Phân tích main.dart
Mở: `lib/main.dart`
void main() {

runApp(const MyApp());
}

- `main()` → điểm bắt đầu
- `runApp()` → gắn widget gốc

⚠️ Flutter KHÔNG chạy từ XML hay HTML

12. BƯỚC 7 – XÂY DỰNG UI PROFILE ĐƠN GIẢN VỚI STATELESSWIDGET

Mục tiêu học thuật: Giúp sinh viên hiểu **StatelessWidget là gì**, cách **tạo widget mới**, **đặt đúng file**, và **gắn widget vào UI để kiểm chứng Hot Reload**.

7.1. TẠO FILE CHO WIDGET MỚI

📂 Thư mục:

```
lib/
└── widgets/
```

👉 Chuột phải → **New File** → đặt tên:

hello_widget.dart

✅ Kiểm chứng:

- File `hello_widget.dart` xuất hiện trong thư mục `widgets`

7.2. VIẾT CODE STATELESSWIDGET

📄 Mở file: `lib/widgets/hello_widget.dart`

📋 Dán code sau:

```
import 'package:flutter/material.dart';

class HelloWidget extends StatelessWidget {
const HelloWidget({super.key});

@override
Widget build(BuildContext context) {
return const Text(
'Hello Flutter',
style: TextStyle(fontSize: 24),
);
}
}
```

📌 Giải thích :

- `StatelessWidget` → UI **không thay đổi theo dữ liệu**
- `build()` → nơi Flutter dựng giao diện

7.3. GẮN HELLOWIDGET VÀO UI

📄 Mở file: `lib/main.dart`

👉 Thêm import ở đầu file:

import 'widgets/hello_widget.dart';

👉 Sửa phần `home:` trong `MaterialApp`:

📌 File gắn widget

lib/main.dart

```
home: Scaffold(
body: Center(
child: HelloWidget(),
),
),
```

✅ Chạy được – Hot Reload OK

📌 Lưu ý :

- `Center` dùng để căn giữa UI
- Widget **phải được gắn vào cây widget** thì mới hiển thị

7.4. KIỂM CHỨNG KẾT QUẢ (HOT RELOAD)

🎯 Thực hiện:

- Nhấn **Ctrl + S** để lưu file

✅ Thành công khi:

- App đang chạy
- Màn hình hiển thị chữ **"Hello Flutter"**

💡 Thử nghiệm thêm:

- Đổi text thành `Hello World`
- Lưu file → UI đổi ngay

📌 Nhấn mạnh kiến thức:

- Đây là **Hot Reload**
- Không cần chạy lại app

⚠️ Lỗi thường gặp:

- Quên import `hello_widget.dart`
- Gõ sai tên class `HelloWidget`
- Không lưu file nên tưởng Hot Reload không hoạt động

👉 Sau bước này, sinh viên đã:

- Tạo được widget riêng
- Hiểu StatelessWidget
- Thấy rõ triết lý **Everything is a Widget**

13. Bước 8 – Counter với StatefulWidget

`lib/widgets/counter_widget.dart`

📋 **Dán toàn bộ code sau**:

```
import 'package:flutter/material.dart';

class CounterWidget extends StatefulWidget {
const CounterWidget({super.key});

@override
```

```
State<CounterWidget> createState() => _CounterWidgetState();
}

class _CounterWidgetState extends State<CounterWidget> {
int count = 0;

@override
Widget build(BuildContext context) {
return Column(
children: [
Text('Count: $count', style: const TextStyle(fontSize: 24)),
const SizedBox(height: 10),
ElevatedButton(
onPressed: () {
setState(() {
count++;
});
},
child: const Text('Increase'),
)
],
);
}
}
```

📌 **Giải thích ngắn**

- `count` → dữ liệu thay đổi
- `setState()` → báo Flutter vẽ lại UI

8.3. KIỂM CHỨNG (CHƯA CHẠY)

⚠️ **CHƯA chạy được ngay**

👉 Vì **chưa gắn widget này vào màn hình**

14. BƯỚC 9 – PROFILE CARD (STATELESSWIDGET)

👉 **Tạo widget hiển thị thông tin cá nhân**

9.1. TẠO FILE MỚI

📂 Vào:

```
lib/
 └── widgets/
```

👉 Chuột phải → New File → đặt tên:

`profile_card.dart`

9.2. MỞ FILE & VIẾT CODE

📄 **Mở file**:

`lib/widgets/profile_card.dart`

📋 **Dán code sau**:

```
import 'package:flutter/material.dart';

class ProfileCard extends StatelessWidget {
const ProfileCard({super.key});

@override
Widget build(BuildContext context) {
return Card(
margin: const EdgeInsets.all(16),
child: Padding(
padding: const EdgeInsets.all(16),
child: Column(
children: const [
CircleAvatar(
radius: 40,
child: Icon(Icons.person, size: 40),
),
SizedBox(height: 10),
Text(
'Nguyễn Văn A',
style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
),
Text('Flutter Beginner'),
],
),
),
);
}
}
```

📌 **Giải thích**

- `StatelessWidget` → dữ liệu không đổi
- Dùng `Card`, `Column`, `Icon`

9.3. KIỂM CHỨNG

⚠️ **Chưa hiển thị**

👉 Phải gắn vào màn hình chính

15. Bước 10 – – HOME SCREEN (KẾT HỢP TẤT CẢ)

👉 Đây là **bước quan trọng nhất**, sinh viên hay nhầm.

10.1. TẠO FILE HOME SCREEN

📂 Vào:

```
lib/
 └── screens/
```

👉 Chuột phải → New File → đặt tên:

`home_screen.dart`

10.2. MỞ FILE & VIẾT CODE

📄 **Mở file**:
`lib/screens/home_screen.dart`

📋 **Dán code sau**:

```
import 'package:flutter/material.dart';
import '../widgets/profile_card.dart';
import '../widgets/counter_widget.dart';

class HomeScreen extends StatelessWidget {
const HomeScreen({super.key});

@override
Widget build(BuildContext context) {
return Scaffold(
appBar: AppBar(title: const Text('My First Flutter App')),
body: Column(
mainAxisAlignment: MainAxisAlignment.center,
children: const [
ProfileCard(),
SizedBox(height: 20),
CounterWidget(),
],
),
);
}
}
```

📌 **Giải thích**

- `HomeScreen` = màn hình chính
- `Scaffold` = khung app
- Import widget bằng đường dẫn tương đối

10.3. GẮN HOME SCREEN VÀO APP

📄 **Mở file**:
`lib/main.dart`

👉 **home :**

```
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';

void main() {
runApp(const MyApp());
}

class MyApp extends StatelessWidget {
const MyApp({super.key});

@override
Widget build(BuildContext context) {
return const MaterialApp(
debugShowCheckedModeBanner: false,
home: HomeScreen(),
);
```

```
  }
}
```

10.4. KIỂM CHỨNG THÀNH CÔNG

🎯 **Kết quả đúng khi:**

- App mở lên
- Thấy:
  - Avatar + tên
  - Counter
- Bấm nút **Increase** → số tăng

❌ **Lỗi thường gặp**

Lỗi	Nguyên nhân
Không thấy UI	Quên gắn HomeScreen
Lỗi import	Sai đường dẫn
UI không đổi	Quên setState

Ctrl + S → Hot Reload

- Thay text → UI cập nhật ngay

👉 Nhấn mạnh: Hot Reload ≠ Hot Restart

16. Các lỗi thường gặp & tips

Lỗi

- RenderFlex overflow → thiếu Expanded / SingleChildScrollView
- Widget quá lớn → vỡ layout

Tips

- Chia widget nhỏ
- Luôn đọc error message
- Format code: `flutter format .`

17. Kết quả đạt được sau dự án

Sinh viên có thể:

- Tự tạo project Flutter
- Hiểu rõ Widget, State, UI update
- Tự debug môi trường
- Sẵn sàng học Module 2 (Layout & Widgets)

18. Bài tập mở rộng (khuyến nghị)

- Thêm nút reset counter
- Thêm theme màu
- Tách ProfileCard nhận dữ liệu qua constructor